

1 Digital Signature Scheme

Digital signature schemes are a 3 PPT algorithms:

1. $\text{KeyGen}(1^k, R) \rightarrow (pk, sk)$
2. $\text{Sign}(pk, sk, D, R) \rightarrow \text{sig}(D)$
3. $\text{Verify}(pk, D, \text{Sig}(D))$

We build a signature scheme that is secure using a 1 way permutation and collision resistant hash function.

1.1 Lamport 1-Time Signature Scheme

Consider the following simple signature scheme using 1-way permutation f to sign bit b .

- Create private keys x_0, x_1 and publish public keys $f(x_0), f(x_1)$.
- To sign bit b , output x_b as the signature for b .
- Verify simply checks that $f(x_b) = b$.

To demonstrate why this is secure, we construct an adversary A that can invert one way permutation f given and an adversary A' that can forge a signature.

1. Challenger C sends $y = f(x), x \leftarrow \$$ to A who needs to generate an x' such that $f(x') = y$.
2. A , given oracle access to A' , imitates that challenger C' by sneding A' public keys $(y, f(x'))$ with $x' \leftarrow \$$. A' will now ask for a signature of either y or $f(x')$ by sending bit b . If $b = 1$
3. If $b = 0$, A starts from the beginning and switches the order of the public keys.
4. If $b = 1$, A reveals signature x' and, A' must be able to forge the signature for $b = 1$ with non negligible probability. Since the only valid signature for 1 would be x , A inverts $f(x)$ with non negligible probability.

1.1.1 Extending to sign multiple bits

An easy extension of the above idea to sign a message of length n would be to create $2n$ (pk, sk) pairs using $x_0, \dots, x_n$ for randomly selected x_i values, and signing each bit in the message by revealing x_{bi} . This signature scheme is secure under the assumption that the adversary sees only one signature. The proof of security follows from the original proof:

1. The adversary A receives $y = f(x)$ for random x .
2. It creates a table of public and private keys with $2n$ entries using random x_{0i}, x_{1i} values.
3. It randomly replaces one $f(x_{0i})$ values with y .
4. If the requested message at index i requests to reveal 0, it fails and retries by selecting a new index and bit value ($f(x_{1i'})$), otherwise it can reveal x_{1b} .
5. If, at index i , A' signs the bit corresponding to $f(x)$, then we successfully invert x .

This succeeds with probability $\epsilon/2n$, which is the advantage of A' multiplied by the probability that it doesn't ask for the bit at index i AND that it differs at index i from the original requested message. The probability $1/n$ comes from the fact that at least one bit in the newly signed message must be different from the original requested message.

1.2 Multi-use PK Signature Scheme

We have the following signature scheme that uses a single public key pk_0 than can sign messages of length k under the assumption that collision resistant hash functions exist.

1. We generate pk_0 by following Lamport's signature scheme to generate a public, private key with $4k$ $(x, f(x))$ pairs.
2. To sign a message of length k , you use the first k bits of the pk to reveal the corresponding bits.
3. For subsequent messages, you generate a new key pk_i , which is verified as being written by the owner of pk_0 by using the second k bits of pk_{i-1} to sign the **hash** of pk_i (which is $4nk$ bits long) to k bits, and now use the first k (pk, sk) pairs of this new key to sign new message m_i

This scheme is secure as one can see it follows from Lamport's signature scheme. Furthermore, to fake a key k_i , one would need to find a collision which can only happen with negligible probability since we use a collision resistant hash function.

1.3 Naor-Yung trees

We introduce an improvement from the prior signature scheme that can sign up to $n^{\log(n)}$ messages. To do this, we use a hash function

$$h : \{0, 1\}^{4n^2} \rightarrow \{0, 1\}^n$$

The idea is to simulate a tree of height $\log^2 n$ and use the *path* from root (the published PK) to the leaf. The i th signature follows from the i 'th leaf node in the tree. To sign message i , we take the path down to the leaf from the root and use that as a signature. Since there is no way to actually have access to this entire tree, we have to simulate its generation. For the first signature, it generates the left and right child of the node containing the root (pk), then signs the reduced hash of both children's public keys concatenated together. If a new node ever needs to be generated (to access a leaf), the node down to the leaf from that node is also generated.

2 Hash Functions

There are three key notions of collision resistance on the family $\mathbb{H}$, the family of hash functions.

Collision Resistance:

$$PR_{H,R}[h \leftarrow H_n : (x_n, y_n) \leftarrow A(h) : h(x_n) = h(y_n)] \leq \frac{1}{n^c}$$

Or the probability that any PPT adversary A can find two distinct elements (x_n, y_n) given hash function h that hash to the same value is negligible

Universal one-way [Naor-Yung]

$$Pr_{H,R}[(x_n, \alpha) \leftarrow A(1^n); h \leftarrow H_n; y_n \leftarrow A(\alpha, h, x_n) : h(x_n) = h(y_n)] \leq \frac{1}{n^c}$$

Given a value x_n generated by A , the challenger provides a randomly selected hash function from the family h . The probability any adversary can produce a value $y_n \neq x_n$ where $h(x_n) = h(y_n)$ is negligible.

Two-Wise Independence

$$Pr_{H,R}[(x_n, y_n) \leftarrow A(1^n); h \leftarrow H_n : h(x_n) = h(y_n)] \leq \frac{1}{n^c}$$

The probability that any two different values hash to the same value given a random hash function from the family is negligible

2.1 Two-Ways Independent Function

Consider the following two-wise independent hash family:

$$H : h_{a,b}(x) = (ax + b)_{1,n}$$

That is, for random a, b values, the hash function takes the $ax + b$ value and removes the first bit. This